

Incorrect expectation values of separate Hamiltonian terms

<https://github.com/qiskit-community/qiskit-nature/issues/806>

ROW 66

Incorrect expectation values of separate Hamiltonian terms #806

[New issue](#)[Closed](#)

MartinBeseda opened on Aug 25, 2022

...

Environment

- Qiskit Nature version: 0.4.4
- Python version: 3.9.7
- Operating system: Kubuntu 20.04.4 LTS

What is happening?

The expectation values obtained via `ExpectationFactory` and `CircuitSampler` classes are incorrect in this case.

How can we reproduce the issue?

The following snippet reproduces the problem, where it can be seen, that evaluation of the expectation value of separate Hamiltonian terms w.r.t. Hartree-Fock state always equals 0, while the expectation value w.r.t. ground state obtained via VQE is incorrect for `+_0 -_0` and `+_2 -_2` terms.

Due to a slightly wordy syntax necessary to obtain expectation values, the functionality itself is enclosed in classes `OperatorEvaluatorBase` and `HermitianOperatorEvaluator`.

```
#!/usr/bin/env python3
from abc import ABC
from typing import Callable, Union
import numpy as np
from qiskit import QuantumCircuit
from qiskit.circuit import Parameter
from qiskit.circuit.parametervector import ParameterVectorElement
from qiskit.opflow import ExpectationFactory, StateFn, CircuitStateFn, CircuitSampler
from qiskit.providers.aer import Aer
from qiskit.utils import QuantumInstance
from qiskit.utils.backend_utils import is_aer_provider
from qiskit_nature.algorithms import GroundStateEigensolver, VQEUCFactory
from qiskit_nature.circuit.library import UCC, HartreeFock
from qiskit_nature.converters.second_quantization import QubitConverter
from qiskit_nature.drivers import Molecule
from qiskit_nature.drivers.second_quantization import ElectronicStructureMoleculeDriver, ElectronicStructureD
from qiskit_nature.mappers.second_quantization import JordanWignerMapper
from qiskit_nature.operators.second_quantization import FermionicOp
from qiskit_nature.problems.second_quantization import ElectronicStructureProblem
from qiskit_nature.transformers.second_quantization.electronic import ActiveSpaceTransformer

class OperatorEvaluatorBase(ABC):
    def __init__(self, operator, backend, include_custom: bool = False) -> None:
        self.backend = backend
        self._expectation_obj = ExpectationFactory.build(operator=operator,
                                                         backend=backend,
                                                         include_custom=include_custom)
        self._measurable_operator = self._expectation_obj.convert(StateFn(operator,
                                                                           is_measurement=True)).reduce()

    def get_evaluation_func(self, circuit: QuantumCircuit) -> Callable:
        pass

class HermitianOperatorEvaluator(OperatorEvaluatorBase):
    def __init__(self, operator, backend, include_custom: bool = False) -> None:
        super(HermitianOperatorEvaluator, self). \
            __init__(operator, backend, include_custom)

    def get_evaluation_func(self, circuit: QuantumCircuit) -> Callable:
        expectation_operator = self._measurable_operator.compose(CircuitStateFn(circuit)).reduce()
        circ_sampler = CircuitSampler(self.backend, param_qobj=is_aer_provider(self.backend))

    def evaluation_func(circ_params: Union[np.array, list, tuple,
                                           dict[Union[ParameterVectorElement, Parameter], float]]) -> flo
```

Assignees

No one assigned

Labels

bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



```

def evaluation_func(circ_params: Union[np.array, list, tuple,
                                     dict[Union[ParameterVectorElement, Parameter], float]]) -> float:
    param_binding = None
    if isinstance(circ_params, dict):
        param_binding = circ_params
    else:
        param_binding = {p: circ_params[i] for i, p in enumerate(circuit.parameters)}
    return circ_sampler.convert(expectation_operator, params=param_binding).eval().real

return evaluation_func

if __name__ == '__main__':

    backend = Aer.get_backend('statevector_simulator')

    geometry = [('N', [0.000000000000, 0.000000000000, 0.000000000000]),
                ('C', [0.000000000000, 0.000000000000, 1.498047000000]),
                ('H', [0.000000000000, -0.938765985000, 2.004775984000]),
                ('H', [0.000000000000, 0.938765985000, 2.004775984000]),
                ('H', [-0.744681452, -0.131307432, -0.634501434])]

    # Obtaining ground state
    driver = ElectronicStructureMoleculeDriver(Molecule(geometry),
                                                basis="sto3g",
                                                driver_type=ElectronicStructureDriverType.PSI4)

    as_transformer = ActiveSpaceTransformer(num_electrons=2, num_molecular_orbitals=2)
    es_problem = ElectronicStructureProblem(driver, transformers=[as_transformer])
    converter = QubitConverter(JordanWignerMapper())

    ansatz = UCC(qubit_converter=converter, num_particles=(1, 1), num_spin_orbitals=4, excitations='sd',
                alpha_spin=True, beta_spin=True, max_spin_excitation=None, generalized=True, preserve_spin=True,
                reps=1, initial_state=None)

    solver = GroundStateEigensolver(converter, VQEUCFactory(quantum_instance=QuantumInstance(backend), ansatz=ansatz))

    ground_state = solver.solve(es_problem)
    optimal_circ = ansatz.bind_parameters(ground_state.raw_result.optimal_parameters)

    # Obtain 1-body terms of Hamiltonian and rewrite the coefficients to 1
    ham_op = es_problem.second_q_ops()[0]
    one_body_terms = [(e[0], 1) for e in ham_op.to_list() if len(e[0].split()) == 2]

    # Obtain expectation values for every term separately
    exp_funcs_vqe = [HermitianOperatorEvaluator(converter.convert(FermionicOp(e)),
                                                backend,
                                                include_custom=True).get_evaluation_func(optimal_circ)
                     for e in one_body_terms]

    # Expectation values for Hartree-Fock state
    hf_circ = HartreeFock(num_spin_orbitals=4,
                        num_particles=(1, 1),
                        qubit_converter=converter)

    exp_funcs_hf = [HermitianOperatorEvaluator(converter.convert(FermionicOp(e)),
                                                backend,
                                                include_custom=True).get_evaluation_func(hf_circ)
                    for e in one_body_terms]

    print('Expectation values for separate 1-body Hamiltonian terms with HF-state circuit:')
    for t, f in zip(one_body_terms, exp_funcs_hf):
        print(f'{t[0]}: {f({})}')

    print('Expectation values for separate 1-body Hamiltonian terms with VQE-ground-state:')
    for t, f in zip(one_body_terms, exp_funcs_vqe):
        print(f'{t[0]}: {f({})}')

```

The output

Expectation values for separate 1-body Hamiltonian terms with HF-state circuit:

```
+_0 _0: 0.0
+_0 _1: 0.0
+_1 _0: 0.0
+_1 _1: 0.0
+_2 _2: 0.0
+_2 _3: 0.0
+_3 _2: 0.0
+_3 _3: 0.0
```

Expectation values for separate 1-body Hamiltonian terms with VQE-ground-state:

```
+_0 _0: 0.08137254743300243
+_0 _1: 0.12130051705413182
+_1 _0: 0.12130051705413182
+_1 _1: 0.08137254743300243
+_2 _2: 0.08137254743300243
+_2 _3: 0.12130051705413182
+_3 _2: 0.12130051705413182
+_3 _3: 0.08137254743300243
```

What should happen?

The expectation values obtained w.r.t. HF state should be nonzeros, while the output for the ground state should look like this:

```
+_0 _0: 0.919251665
+_0 _1: 0.12130051705413182
+_1 _0: 0.12130051705413182
+_1 _1: 0.08137254743300243
+_2 _2: 0.919251665
+_2 _3: 0.12130051705413182
+_3 _2: 0.12130051705413182
+_3 _3: 0.08137254743300243
```

Any suggestions?

If it helps, I can write more specific and brief example - this one was created with an intention of demonstrating the problem in the full context.



MartinBeseda added **bug** on Aug 25, 2022



mrossinek on Aug 25, 2022

Member ...

The title of your issue seems to suggest that this is only dealing with aspects of Qiskit Terra, rather than Nature. Please produce a minimal example which concisely summarizes the issue you are trying to seek out here.



mrossinek added **status: need informati...** on Aug 25, 2022



MartinBeseda on Aug 26, 2022

Author ...

@mrossinek I put the problem here, as I'm trying to obtain the expectation values of separated Hamiltonian terms via the abovementioned classes and I'm obtaining incorrect results even in simple cases.

To be specific, e.g. I want to take the first term of Hamiltonian `+_0 _0` and obtain its expectation value with a Hartree-Fock state, but I'm obtaining zero. That's the same for all the one-body operators. Are they not supposed to be used like this?

I've tried to simplify the code to focus on this one case only, without the ground state results.

```
#!/usr/bin/env python3
from qiskit.opflow import ExpectationFactory, StateFn, CircuitStateFn, CircuitSampler
from qiskit.providers.aer import Aer
from qiskit.utils.backend_utils import is_aer_provider
from qiskit_nature.circuit.library import HartreeFock
```

```

from qiskit_nature.circuit.library import HartreeFock
from qiskit_nature.converters.second_quantization import QubitConverter
from qiskit_nature.drivers import Molecule
from qiskit_nature.drivers.second_quantization import ElectronicStructureMoleculeDriver, ElectronicStructureDriver
from qiskit_nature.mappers.second_quantization import JordanWignerMapper
from qiskit_nature.operators.second_quantization import FermionicOp
from qiskit_nature.problems.second_quantization import ElectronicStructureProblem
from qiskit_nature.transformers.second_quantization.electronic import ActiveSpaceTransformer

if __name__ == '__main__':

    backend = Aer.get_backend('statevector_simulator')

    geometry = [('N', [0.000000000000, 0.000000000000, 0.000000000000]),
                ('C', [0.000000000000, 0.000000000000, 1.498047000000]),
                ('H', [0.000000000000, -0.938765985000, 2.004775984000]),
                ('H', [0.000000000000, 0.938765985000, 2.004775984000]),
                ('H', [-0.744681452, -0.131307432, -0.634501434])]

    driver = ElectronicStructureMoleculeDriver(Molecule(geometry),
                                              basis="sto3g",
                                              driver_type=ElectronicStructureDriverType.PSI4)

    as_transformer = ActiveSpaceTransformer(num_electrons=2, num_molecular_orbitals=2)
    es_problem = ElectronicStructureProblem(driver, transformers=[as_transformer])
    converter = QubitConverter(JordanWignerMapper())

    # Obtain 1-body terms of Hamiltonian and rewrite the coefficients to 1
    ham_op = es_problem.second_q_ops()[0]
    one_body_terms = [(e[0], 1) for e in ham_op.to_list() if len(e[0].split()) == 2]

    # Expectation values for Hartree-Fock state
    hf_circ = HartreeFock(num_spin_orbitals=4,
                        num_particles=(1, 1),
                        qubit_converter=converter)

    op = converter.convert(FermionicOp(one_body_terms[0]))

    measurable_op = ExpectationFactory.build(operator=op,
                                             backend=backend,
                                             include_custom=True).convert(StateFn(op, is_measurement=True)).real

    expec_op = measurable_op.compose(CircuitStateFn(hf_circ)).reduce()

    circ_sampler = CircuitSampler(backend, param_qobj=is_aer_provider(backend))
    print(f'⟨one_body_terms[0][0]⟩ = {circ_sampler.convert(expec_op).eval().real}')

```



MartinBeseda changed the title ~~Strange behavior of "Expectation" combined with "CircuitSampler"~~ to **Incorrect expectation values of separate Hamiltonian terms** on Aug 26, 2022



mrossinek on Aug 26, 2022

Member ...

TL;DR: supply the correct `register_length` value when initializing your `FermionicOp` instances, e.g. like so:

```
FermionicOp(one_body_terms[0], register_length=4)
```



Explanation

Alright the issue which you are running into here is two-fold:

1. You are initializing your custom `FermionicOp` instances using the sparse label notation without providing a value for `register_length`. This behavior results in an automatic computation of this register length which is why this does not raise an error. What this means specifically for you, is that the fermionic operators have length 1 which does not match your expectation of length 4. You can see this when printing e.g. `op` which is the resulting qubit operator, also only of length 1.

```
circ_sampler = CircuitSampler(backend, param_qobj=is_aer_provider(backend))
print(f'⟨one_body_terms[0][0]⟩ = {circ_sampler.convert(expect_op).eval().real}')
```



MartinBeseda changed the title ~~Strange behavior of Expectation combined with CircuitSampler~~ Incorrect expectation values of separate Hamiltonian terms on Aug 26, 2022



mrossinek on Aug 26, 2022

Member ...

TL;DR: supply the correct `register_length` value when initializing your `FermionicOp` instances, e.g. like so:

```
FermionicOp(one_body_terms[0], register_length=4)
```



Explanation

Alright the issue which you are running into here is two-fold:

1. You are initializing your custom `FermionicOp` instances using the sparse label notation *without* providing a value for `register_length`. This behavior results in an automatic computation of this register length which is why this does not raise an error. What this means specifically for you, is that the fermionic operators have length 1 which does not match your expectation of length 4. You can see this when printing e.g. `op` which is the resulting qubit operator, also only of length 1.
2. The second problem is endianess. Terra orders their qubits like computer scientists, in little endian format. On the contrary, in Nature our operators are big endian, since is the more wide-spread standard in chemistry/science-oriented software packages. This is the reason, why the above is actually causing issues for you at all, since the the HartreeFock state you initialize, has no X applied on the "last" qubit (from Nature's point of view), which is where Terra will apply the single-length qubit operator evaluation (since it considers this the "first" qubit).

This has been brought up before here: [#609](#)



mrossinek closed this as `completed` on Aug 26, 2022

mrossinek removed `status: need informati...` on Aug 26, 2022